

Islamic University of Technology

Department of Computer Science and Engineering (CSE)

Software Engineering Programme

SWE 4701: Software Metrics and Process

Research Review Report

Automated Code-centric Software Vulnerability Assessment: How Far Are We? An Empirical Study in C/C++

Anh The Nguyen; Triet Huynh Minh Le; M. Ali Babar (ESEM 2024)

Submitted by

Antara Arifa Mullick
ID: 210042132
B.Sc. in Software Engineering
Department of CSE

Submission Date

12 March 2026

1 Summary of the Paper

Software vulnerabilities in C/C++ can lead to unauthorized access, data leakage, or system failure. Because developers cannot fix all vulnerabilities at once, severity-based prioritization is essential. The Common Vulnerability Scoring System (CVSS) [4] provides standardized severity scores, but these are only published *after* public disclosure in repositories such as the National Vulnerability Database [5]. A concrete example from the paper is CVE-2016-2315, patched 185 days before its public disclosure. During that gap, developers had no severity information to act on.

Nguyen, Le, and Babar [1] address this by predicting CVSS metrics *directly from vulnerable source code functions*, removing the dependency on published reports. This is the first empirical study to do so for C/C++, the language family with the highest number of reported CVEs. The study has two research questions:

- **RQ1:** How well do multi-class ML and DL models independently predict each of the seven CVSSv2 Base metrics from function-level source code?
- **RQ2:** Can multi-task deep learning—one model predicting all seven metrics simultaneously—improve both effectiveness and efficiency over the multi-class approach?

Together, these questions allow the paper to compare not only predictive accuracy but also the practical trade-off between effectiveness and computational cost. The dataset is derived from Big-Vul [2], filtered to **9,993 vulnerable C/C++ functions** labeled with seven CVSSv2 Base metrics: Access Vector, Access Complexity, Authentication, Confidentiality, Integrity, Availability, and Severity. Six ML classifiers (LR, SVM, KNN, RF, XGB, LGBM) are evaluated with four feature extraction methods (BoT, BoST, Word2Vec, fastText), and five DL architectures (CNN, LSTM, GGNN, GCN, CodeBERT [3]) are tested in both multi-class and multi-task configurations.

The three headline findings are: (1) LGBM with Bag-of-Subtokens (LGBM+BoST, MCC = 0.680) slightly outperforms the best multi-class DL model, CodeBERT (MCC = 0.673), while training *four times faster*; (2) switching all DL models to multi-task learning improves MCC by 8–22%; and (3) multi-task CodeBERT achieves the best overall result (MCC = 0.734, F1 = 0.820), outperforming LGBM+BoST by 8%.

2 Strengths and Weaknesses

Strengths. This paper addresses a genuine and underexplored research gap. Function-level SV *detection* has been widely studied, but function-level SV *assessment* in C/C++ had never been empirically investigated. By shifting the focus to severity prediction from code, the paper makes a useful contribution to early vulnerability triage. The evaluation is comprehensive: 11 models, 4 feature methods, and 7 prediction targets produce over 44 experimental configurations. The multi-task learning contribution is practically valuable—a single unified model cuts training time by 23% while improving accuracy over seven separate models. Reproducibility is fully supported through the public release of data, code, and models [6]. The paper also aligns with prior SV detection findings (the same Transformer $>$ Graph $>$ Sequence DL ranking), which strengthens credibility through external consistency.

Weaknesses. The scope is limited to C/C++, so conclusions may not transfer to Java, Python, or other languages. The exclusive use of CVSSv2 is increasingly outdated—CVSSv3 (2015)

and CVSSv4 (2023) are now the dominant industry standards [4]. The deliberate exclusion of class-imbalance remediation isolates model-choice effects but leaves some metrics poorly handled in practice (Authentication has 98% of instances in a single class). Most critically, no statistical significance tests are reported anywhere. All model comparisons are raw point estimates, making it impossible to determine which differences are meaningful versus within normal variance.

3 Critical Evaluation of the Methodology

Dataset construction. Big-Vul provides a large, diverse foundation spanning 348 GitHub projects and 91 vulnerability types from 2002 to 2019 [2]. Labeling functions modified inside vulnerability-fixing commits as vulnerable is the standard and traceable approach. Access Vector, absent from Big-Vul, was retrieved via CVE-IDs from NVD—pragmatic, but with a small risk of metadata mismatch. Severity scores were re-discretized into Low/Medium/High per NVD qualitative ratings, consistent with prior work.

Two deeper concerns remain. First, no *temporal split* is used: the random 80/10/10 division may place functions from the same CVE or repository in both training and test sets, risking data leakage through project-specific code patterns. Second, the paper does not report how many functions were removed during the October 2023 GitHub availability check, nor whether the removal introduced systematic bias toward more recently maintained projects.

Model design. ML configurations follow established prior work, providing methodological continuity. The multi-task DL design is clean: only the output layer is modified to include seven classification heads, with the shared encoder left unchanged, enabling a fair comparison with the multi-class baseline. The multi-task loss is the mean of seven per-task cross-entropy values:

$$\mathcal{L} = \frac{1}{7}\sum_{n=1}^{7} \ell_n, \quad \ell_n = -w_n \left[y_n \cdot \log \sigma(x_n) + (1 - y_n) \cdot \log(1 - \sigma(x_n)) \right]$$

This uses equal weights w_n across all tasks. Given the large imbalance differences among metrics, equal weighting may under-penalize minority-class errors. Class-weighted or uncertainty-based loss formulations would be a natural improvement.

Evaluation protocol. Stratified splitting and inter-set deduplication are both sound practices. Hyperparameter tuning is well-documented: grid search for ML and Ray Tune with randomized grid search for DL, with model selection based on validation MCC. The main weakness is reliance on a *single split* with no k-fold cross-validation, repeated runs, or bootstrap resampling. Without these, result variance is unknown and reported MCC values should be treated as point estimates rather than stable benchmarks.

4 Detailed Analysis of Results

4.1 Metrics and Measures Used

Three performance indicators are used. **MCC** is the primary metric, ranging from -1 to $+1$, computed from all four confusion matrix cells (TP, TN, FP, FN). It is reliable under severe class imbalance—the correct choice here given the skewed CVSS distributions. **Macro F1-Score**

complements MCC by averaging per-class F1 values, giving equal weight to rare and common classes alike. **Training time** in seconds is reported as an efficiency proxy, directly supporting the paper's practical recommendations. However, inference time and memory footprint are not reported—both matter for deployment in CI/CD pipelines where latency and compute budgets are constrained.

4.2 Analyses Conducted

RQ1 — Multi-class ML vs. DL. Ensemble ML classifiers consistently outperform non-ensemble ones, with LGBM achieving the best average MCC of 0.643. Sub-token features (BoST, fastText) outperform token-level features because breaking tokens into character sub-sequences captures code morphology and improves generalization to unseen identifiers. The best combination, LGBM+BoST (MCC = 0.680, F1 = 0.763), outperforms all five multi-class DL models on 6 of 7 CVSS metrics. The exception is Access Vector—the most imbalanced metric (78% "Network", 1.2% "Adjacent Network")—where DL's richer representations better handle the minority class.

Among DL architectures: CodeBERT (0.673) $>$ GGNN (0.611) $\approx$ GCN (0.608) $>$ LSTM (0.601) $\approx$ CNN (0.596). This Transformer $>$ Graph $>$ Sequence ranking mirrors prior SV detection findings, adding cross-task credibility. The efficiency gap is sharp: CodeBERT requires $\approx$12,063 seconds of training versus $\approx$3,058 seconds for LGBM+BoST—a $4\times$ difference—despite delivering lower MCC in the multi-class setting.

RQ2 — Multi-task Deep Learning. Multi-task learning improves every DL architecture: CNN +22%, LSTM +14%, GGNN +14%, GCN +17%, CodeBERT +9%. Gains are largest for the most imbalanced metrics—Access Complexity (+48%), Authentication (+17%), Confidentiality (+17%)—suggesting that shared encoder representations compensate for minority-class data scarcity. Multi-task CodeBERT (MCC = 0.734, F1 = 0.820) cuts total training time by 23% relative to seven separate CodeBERT models, since encoder weights are updated only once per gradient step.

4.3 Are the Results Sufficient to Support the Conclusions?

The paper draws three main conclusions, assessed separately.

Conclusion 1: LGBM+BoST is competitive with multi-class DL. Strongly supported. LGBM+BoST achieves a higher MCC than CodeBERT (0.680 vs. 0.673) at 75% less training cost. This is consistent with prior software engineering findings that well-tuned ensemble models can match DL on structured tasks.

Conclusion 2: Multi-task DL improves over multi-class DL. Robustly supported. The improvement holds across all five architectures and nearly all metrics, providing strong internal consistency. The pattern—largest gains where imbalance is worst—also offers a coherent mechanistic explanation.

Conclusion 3: Multi-task CodeBERT is definitively the best model. Weakest claim. The 8% MCC advantage (0.734 vs. 0.680) rests on a single random split with no confidence intervals. Without a Wilcoxon signed-rank test or similar, it is impossible to confirm this difference is statistically meaningful rather than within expected split-to-split variance.

4.4 Validation of Results

The study performs only internal validation on a single held-out test split. There is no external validation on independent projects, no cross-project evaluation, and no temporal holdout. Stratification and deduplication are correct and necessary steps, but they do not substitute for cross-validation. The reported MCC and F1 values are best understood as upper-bound estimates rather than reliable deployment benchmarks. Table 1 summarizes the key reported numbers.

Table 1: Summary of key performance results reported in the paper

Model / Setting	**MCC**	**F1**	**Note**
LGBM + BoST (multi-class ML)	0.680	0.763	Best ML model
CodeBERT (multi-class DL)	0.673	0.746	Best multi-class DL
CodeBERT (multi-task DL)	0.734	0.820	Best overall model

5 Threats to Validity

Threats acknowledged by the authors include: (1) implementation fidelity of reproduced models, mitigated by reusing validated packages; (2) soundness of the multi-task modification, addressed by changing only output layers; (3) incomplete hyperparameter search, accepted as a practical constraint; and (4) limited generalizability beyond C/C++, partially offset by dataset diversity across 300+ projects.

Additional threats not fully addressed:

Label noise. CVSSv2 scores are human-assigned by NVD analysts and are known to be inconsistent across scorers and over time. Some vulnerability scores have been revised post-publication. The impact of this noise on model training and evaluation is not analyzed.

Data leakage via random split. Functions from the same CVE or repository may appear in both training and test sets. Code from the same project shares stylistic and structural patterns, which could allow models to exploit project-level memorization rather than learning genuinely generalizable features.

Absence of statistical significance testing. All comparisons are point estimates. Without tests such as the Wilcoxon signed-rank or Scott-Knott ESD test, it is impossible to confirm which performance differences are statistically reliable versus attributable to split randomness.

Static code representation only. All models operate on tokenized source code. Deeper program properties—control flow, data flow, inter-procedural dependencies—that directly influence exploitability are not captured by the non-graph models.

6 Suggestions for Future Research

- **Upgrade to CVSSv3/v4.** Replicating the study with newer CVSS versions would align it with current industrial practice and test whether the ML–DL trade-off persists under finer-grained scoring schemes.

- **Evaluate large language models.** Instruction-tuned LLMs such as Code Llama, StarCoder, and GPT-4 represent the current state of the art for code-related tasks and should be included as baselines in future work.
- **Cross-language and cross-project generalizability.** Extending the dataset to Java, Python, and Rust with project-stratified splits would empirically test generalizability rather than assuming it from a comparison with a single prior Java study.
- **Temporal evaluation.** Training on vulnerabilities before 2017 and testing on 2017–2019 would simulate realistic deployment conditions and yield more conservative performance estimates.
- **Explainability.** Incorporating SHAP values or attention visualization to identify which code tokens drive CVSS predictions would improve practitioner trust and support model auditing.
- **Class-imbalance remediation.** Investigating class-weighted losses or SMOTE-based augmentation for heavily skewed metrics such as Authentication would address a key practical limitation of the current work.
- **Statistical rigor as standard.** Future studies should adopt k-fold cross-validation, repeated splits, and formal significance testing so that model comparisons carry inferential rather than merely descriptive weight.

7 Personal Reflection

This paper is valuable because it moves vulnerability assessment closer to where developers actually need it: the source code itself. Rather than waiting for a public report, the approach estimates severity directly from vulnerable functions, which is exactly the early-stage support that modern secure development pipelines require.

The most compelling finding for me is that a carefully tuned ML model remained competitive with deep learning. LGBM+BoST slightly outperforms multi-class CodeBERT while being four times cheaper to train. This challenges the assumption that deeper is always better and has real implications for teams in CI/CD environments where frequent model retraining is necessary and compute is limited.

The multi-task result is also intellectually convincing. Consistent improvement across all DL architectures—not just CodeBERT—suggests that CVSS metrics are genuinely correlated at the representation level. A vulnerability that is remotely exploitable tends to also have broader impact, and the shared encoder appears to pick up this structure implicitly.

My main criticism is the lack of statistical rigor. Point estimates from a single split are insufficient to confidently rank models that differ by only a few MCC points. This should be a baseline requirement for studies of this type going forward. Despite this, the paper is well-motivated, reproducible, and practically useful—a solid empirical foundation for future automated vulnerability assessment research.

8 Conclusion

This paper makes a meaningful contribution by demonstrating that software vulnerability severity can be predicted directly from C/C++ source code, without relying on delayed public re-

ports. The key takeaway is practical and nuanced: traditional ML, specifically LGBM with Bag-of-Subtokens, is a strong and efficient baseline that matches multi-class CodeBERT at a fraction of the training cost. When accuracy is the priority, multi-task CodeBERT offers the best overall performance by jointly predicting all seven CVSS metrics through shared representations. The study is reproducible, well-structured, and grounded in real-world vulnerability data, making it a solid benchmark for future research. Its main limitation is the lack of statistical validation, which should be addressed in follow-up work to make model comparisons more conclusive. Overall, the paper succeeds both as an empirical comparison study and as a practical guide for selecting vulnerability assessment models under different resource constraints.

References

[1] A. T. Nguyen, T. H. M. Le, and M. A. Babar, "Automated Code-centric Software Vulnerability Assessment: How Far Are We? An Empirical Study in C/C++," in *Proc. ESEM '24*, Barcelona, Spain, 2024. https://doi.org/10.1145/3674805.3686670

[2] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries," in *Proc. MSR '20*, 2020, pp. 508–512.

[3] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," *arXiv:2002.08155*, 2020.

[4] FIRST, "Common Vulnerability Scoring System," https://www.first.org/cvss/

[5] NIST, "National Vulnerability Database," https://nvd.nist.gov/

[6] A. T. Nguyen, T. H. M. Le, and M. A. Babar, "Reproduction Package," GitHub, 2024. https://github.com/anh56/dl4sa